

Introduction to AI & Data Science

Task 1 - Heuristic Search

Group 2 - Team C (Annas Sheikh, Maid Kanuric, Zafer Sarioglu)

Table of Contents

1. Introduction.....	3
2. Software Architecture.....	4
3. Implementation.....	5
3.1 Functions Overview.....	5
3.2 Function Documentation.....	6
main.calculateCosts (possible_states, heuristic, g_cost).....	6
main.generateRandomSolvableBoard ()......	6
main.get_user_configuration ()......	6
main.hamming (current_state).....	7
main.is_solvable (start_state).....	7
main.manhattan (current_state).....	7
main.neighbors (current_state).....	7
main.run_benchmark (games_to_generate = 100, heuristics = ["manhattan", "hamming"]).....	8
main.solve_puzzle (start_state, heuristic_name).....	8
main.state_to_string (state).....	9
4. Design Decisions.....	10
4.1 Overview.....	10
4.2 Search Algorithm.....	10
4.3 Heuristic Functions.....	10
4.4 Data Structures.....	11
4.4.1 Priority Queue (Open Set).....	11
4.4.2 Closed Set (Visited States).....	11
4.4.3 State Representation.....	12
4.5. Benchmark System.....	12
4.5.1 Workflow for benchmarking.....	12
4.5.2 Collected Informations.....	13
4.5.3 Statistical Output.....	13
4.5.4 Benchmarking Safety Limit.....	13
5. Results & Benchmark.....	14
6. Conclusion.....	14

1. Introduction

This project was completed as part of the course *Introduction to AI and Data Science* at Hochschule Campus Wien, within Task 1 – Heuristic Search. The goal of this assignment was to implement and evaluate the A* search algorithm using the 8-Puzzle problem as an example.

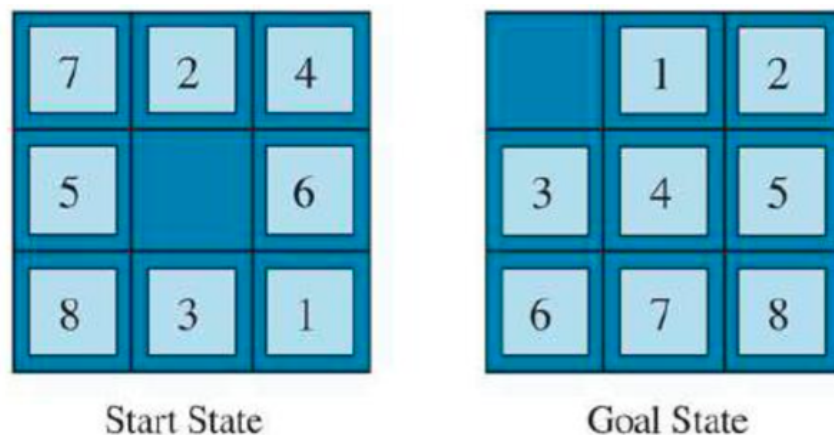
The 8-Puzzle consists of a 3×3 grid with eight numbered tiles and one empty space. The objective is to reach a defined goal configuration by moving the tiles into the correct order. To solve this efficiently, the A* algorithm is used — an informed search method that combines the actual path cost with a heuristic estimate of the remaining distance to the goal.

Two heuristic functions were implemented and compared: the Hamming distance, which counts the number of misplaced tiles, and the Manhattan distance, which sums the distances of the tiles from their correct positions. The program was executed via a terminal interface, where multiple solvable start configurations were generated and benchmarked automatically to compare the performance of both heuristics.

The results demonstrate how the choice of heuristic directly affects the efficiency of the search process and provide a practical understanding of heuristic-based problem solving in artificial intelligence.

The software created as part of this Task is available under the following public git repository:

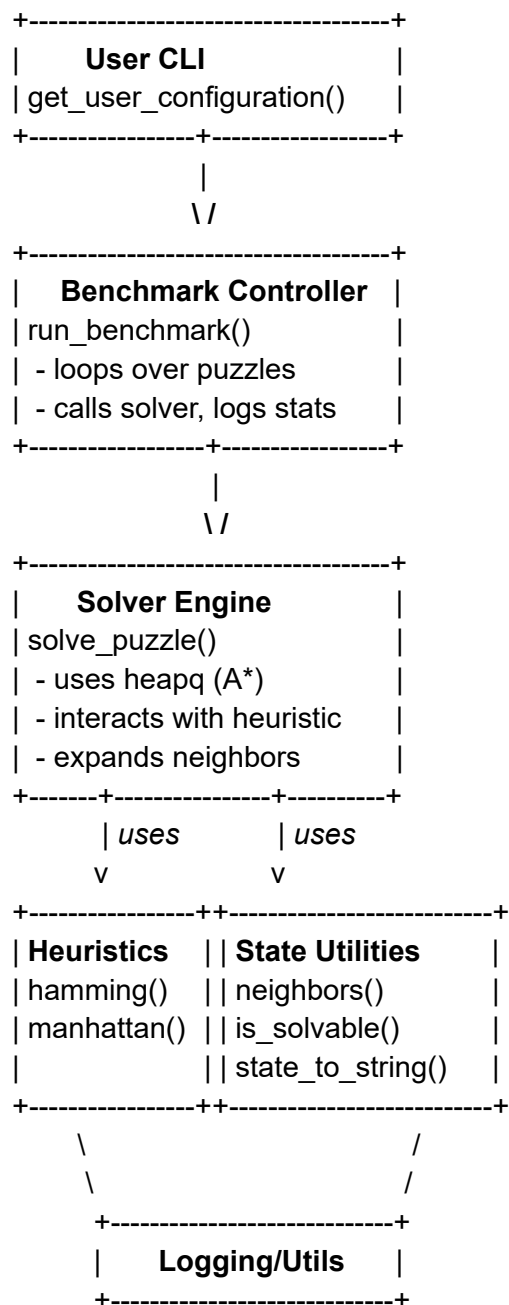
https://github.com/KaMa-0/INTRO_AI_Task_1



Graphical depiction of possible random start state and required goal state, as presented in Task 1.

2. Software Architecture

The following is an illustration of the implemented Software Architecture, showcasing the interactions and flow of the various software components, forming the complete System:



3. Implementation

3.1 Functions Overview

- **hamming** (current_state)

Compute the Hamming distance heuristic.

- **manhattan** (current_state)

Compute the Manhattan distance heuristic.

- **neighbors** (current_state)

Generate all possible neighboring states for a given configuration.

- **calculateCosts** (possible_states, heuristic, g_cost)

Compute total A cost $f(s) = g(s) + h(s)$ for each successor state.*

- **state_to_string** (state)

Convert a puzzle state to a string key.

- **is_solvable** (start_state)

Determine if a puzzle configuration is solvable.

- **generateRandomSolvableBoard** ()

Generate a random solvable 8-puzzle configuration.

- **solve_puzzle** (start_state, heuristic_name)

Solve the 8-puzzle using the A search algorithm.*

- **run_benchmark** (games_to_generate=100, heuristics=["manhattan", "hamming"])

Run benchmark tests comparing heuristics on randomly generated puzzles.

- **get_user_configuration** ()

Interactive CLI for selecting benchmark configuration.

3.2 Function Documentation

main.calculateCosts (possible_states, heuristic, g_cost)

Compute total A* cost $f(s) = g(s) + h(s)$ for each successor state.

Uses either the Manhattan or Hamming heuristic depending on the parameter.

Parameters

<i>possible_states</i>	puzzle states to evaluate.
<i>heuristic</i>	heuristic ("manhattan" or "hamming").
<i>g_cost</i>	of the parent state.

Returns

list List of tuples (state, f_cost) for each possible state.

Note

Terminates the program with exit code 11 if an invalid heuristic is passed.

Definition at line **146** of file [main.py](#).

main.generateRandomSolvableBoard ()

Generate a random solvable 8-puzzle configuration.

Randomly shuffles the tiles until a solvable arrangement is found.

Returns

list A random 3x3 solvable board (list of lists).

Note

The function loops until a solvable board is generated.

Definition at line **216** of file [main.py](#).

main.get_user_configuration ()

Interactive CLI for selecting benchmark configuration.

Asks the user to input the number of puzzles to generate and to select which heuristics to benchmark. Clears the terminal between selections.

Returns

tuple (games_to_generate, heuristics)

- games_to_generate (int): Number of puzzles to benchmark.
- heuristics (list of str): Selected heuristic names.

Definition at line **462** of file [main.py](#).

main.hamming (current_state)

Compute the Hamming distance heuristic.

Counts how many tiles are misplaced compared to the goal state, excluding the blank tile (0).

Parameters

<i>current_state</i>	board represented as a list of lists.
----------------------	---------------------------------------

Returns

int Number of tiles not in their correct position.

Definition at line **38** of file [main.py](#).

main.is_solvable (start_state)

Determine if a puzzle configuration is solvable.

Uses inversion counting: a puzzle is solvable if the number of inversions (pairs of tiles in the wrong order) is even.

Parameters

<i>start_state</i>	board represented as a list of lists.
--------------------	---------------------------------------

Returns

bool True if the puzzle is solvable, False otherwise.

Definition at line **187** of file [main.py](#).

main.manhattan (current_state)

Compute the Manhattan distance heuristic.

Calculates the sum of the absolute distances between each tile's current position and its target position in the goal state, excluding the blank.

Parameters

<i>current_state</i>	board represented as a list of lists.
----------------------	---------------------------------------

Returns

int Sum of Manhattan distances for all tiles.

Definition at line **62** of file [main.py](#).

main.neighbors (current_state)

Generate all possible neighboring states for a given configuration.

Determines all valid puzzle states reachable by moving the blank tile (0) up, down, left, or right.

Parameters

<i>current_state</i>	board represented as a list of lists.
----------------------	---------------------------------------

Returns

list List of possible next 3x3 states.

Definition at line 87 of file [main.py](#).

main.run_benchmark (games_to_generate = 100, heuristics = ["manhattan", "hamming"])

Run benchmark tests comparing heuristics on randomly generated puzzles.

Generates a specified number of random solvable boards and solves each using the selected heuristics (Manhattan and/or Hamming). Logs results and prints statistical summaries (mean and standard deviation of nodes expanded and runtime).

Parameters

<i>games_to_generate</i>	random puzzles to generate and solve.
<i>heuristics</i>	heuristic names to use ("manhattan", "hamming").

Returns

None

Note

Creates log entries for each puzzle in ../log/.

Definition at line 343 of file [main.py](#).

main.solve_puzzle (start_state, heuristic_name)

Solve the 8-puzzle using the A* search algorithm.

Implements A* with a priority queue (heapq) and either the Manhattan or Hamming heuristic. Maintains parent mappings for optional path reconstruction.

Parameters

<i>start_state</i>	puzzle configuration.
<i>heuristic_name</i>	the heuristic ("manhattan" or "hamming").

Returns

tuple (success, iterations, nodes_expanded, elapsed_time)

- success (bool): True if a solution was found.
- iterations (int): Number of A* loop iterations executed.
- nodes_expanded (int): Number of states expanded.
- elapsed_time (float): Runtime in seconds.

Note

Returns (False, ..., ...) if no solution is found or max iteration limit is exceeded.

Definition at line 251 of file [main.py](#).

main.state_to_string (state)

Convert a puzzle state to a string key.

Converts a 3x3 board into a flattened string (e.g., "012345678") used for hashing and state comparison.

Parameters

<i>state</i>	board represented as a list of lists.
--------------	---------------------------------------

Returns

str Linear string representation of the state.

Definition at line **170** of file [main.py](#).

4. Design Decisions

4.1 Overview

The 8-Puzzle solver was implemented in Python using the **A* search algorithm** with two different heuristic functions. The implementation consists of two main components:

1. **Core Algorithm Module ([main.py](#))** - Contains the A* search implementation, heuristic functions, and benchmark logic
2. **Logging System (integrated in [main.py](#))** - Records detailed execution information for debugging and analysis

The system generates random solvable puzzle configurations, solves them using both heuristics, and collects statistical data on performance metrics. Subtasks of the system are recorded in log-files in case further analysis for the calculations becomes necessary.

4.2 Search Algorithm

At the heart of the system, the famous A* algorithm was implemented in order to find the best, least-cost path from a given random `start_state` to the desired `goal_state`.

Why the “A* Algorithm”?

Rationale:

- Guaranteed to find the shortest path from `start_state` to `goal_state`
- Completeness: if a solution exists, it will definitely be found
- Very efficient in terms of time complexity
- Guided by heuristic functions which are at the very core of this Tasks requirements

4.3 Heuristic Functions

Two heuristic functions were implemented to guide the A* search, these were selected on the basis of the Task provided, so no arbitrary/voluntary selection was made, although the Command Line Interface allows for the selection and deselection of either/or heuristic function to be used during any given run.

The two heuristic functions implemented are:

- **Hamming Distance** (counts the number of tiles that are not in their goal position, excluding the blank tile)
- **Manhattan Distance** (calculates the sum of the horizontal and vertical distances of each tile from its goal position)

4.4 Data Structures

4.4.1 Priority Queue (Open Set)

The open set is implemented using Python's "heapq" library, which provides an efficient way to find minimum value in a list.

```
# Priority queue: stores (f_cost, counter, state_string, state, g_cost)
open_heap = []
heapq.heappush(open_heap, (f_cost, 0, start_string, start_state, g_cost))
```

Components:

- **f_cost:** Total estimated cost $f(s) = g(s) + h(s)$
- **counter:** Tie-breaking value to maintain FIFO order for equal f-costs
- **state_string:** String representation for fast comparison, we have an additional function for converting our states to strings.
- **state:** The actual 3×3 board configuration
- **g_cost:** Actual path cost from start to current state

Why heapq instead of Manual Priority Queue?

Rationale:

- Well-tested and reliable
- Simpler code maintenance
- 50-100x faster results

4.4.2 Closed Set (Visited States)

Tracking visited states was also very important for this application. The closed set is implemented as a "Python set()" containing string representations of visited states.

```
# Track visited states
closed_set = set()

# Mark as visited
closed_set.add(current_string)
```

Purpose:

- Prevents revisiting already explored states
- Ensures that there is no infinite loop.

4.4.3 State Representation

States are represented in two forms:

1. Board Representation: 3x3 list of lists

```
# definition of the goal state, as defined in task slides on Moodle
goal_state = [[ 0, 1, 2 ],
              [ 3, 4, 5 ],
              [ 6, 7, 8 ]]
```

2. String Representation: Converted to string for faster comparison

```
def state_to_string(state):
    """
    Convert state to string for comparison.

    Input: state - 3x3 board
    Output: string representation (e.g., "012345678")
    Function: Converts board to string for fast comparison
    """
    result = ""
    for row in state:
        for tile in row:
            result += str(tile)
    return result
```

Why String Representation for States?

Rationale:

- Fast hashing for set operations
- Efficient equality comparison
- Reduced memory overhead compared to storing full board copies
- Easier to work with in some situations and use-cases

4.5. Benchmark System

The benchmark system evaluates both heuristics across multiple random puzzles (100 puzzles in our case, which is also configured as the default value in case not further specified):

4.5.1 Workflow for benchmarking

1. **Generate Random Solvable Board:** Create random configuration and verify solvability
2. **Solve with Selected Heuristics:** Run A* with each chosen heuristic
3. **Collect Metrics:** Record nodes expanded, iterations, and execution time
4. **Statistical Analysis:** Calculate mean and standard deviation

4.5.2 Collected Informations

For each puzzle solution:

1. **Success:** Boolean indicating if solution was found
2. **Iterations:** Number of loop cycles in A* algorithm
3. **Nodes Expanded:** Number of states removed from open set and added to closed set
4. **Execution Time:** Wall-clock time in seconds

4.5.3 Statistical Output

After all puzzles are completed, there is a statistical output, which shows a comparison between the results of the executed Manhattan Heuristic and the Hamming Heuristic.

```
=====
RESULTS - MANHATTAN HEURISTIC
=====
Solved: 100/100
Average nodes expanded: 2154.90 ± 2549.12
Average execution time: 0.0489s ± 0.0611s

=====
RESULTS - HAMMING HEURISTIC
=====
Solved: 100/100
Average nodes expanded: 21959.07 ± 23690.96
Average execution time: 0.5160s ± 0.5843s
```

4.5.4 Benchmarking Safety Limit

As a safeguard a maximum number of iterations was defined with a value of 100.000 (one hundred thousand) iterations. In most cases the algorithm will find a solution in less iterations.

Rationale:

- Prevents infinite loops on implementation bugs
- Protects against extremely difficult random configurations

5. Results & Benchmark

Creating 100 random board start-states and letting the implementation solve those boards using both heuristic functions to determine the lowest cost, the following was observed in terms of memory usage and computation time when comparing the two:

Heuristic Function (used for cost evaluation, to guide the A* algorithm)	Memory Usage (number of expanded nodes in search tree)		Computation Time (time effort to get from start- to goal state)	
	mean	std. deviation	mean	std. deviation
Manhattan	1501.53	1698.00	0.0267s	0.0306s
Hamming	14913.59	16415.98	0.2878s	0.3288s

From the above comparison between the two used heuristics, we can observe the following result:

On average, the **Hamming** Heuristic Function has **approximately 10x the memory usage and computation time** of the **Manhattan** Heuristic Function.

Our testing shows, that between the two heuristics assessed (in the use case of 8-Puzzle solving, using the A* algorithm) the Manhattan Distance is the preferred method of cost calculation and consistently yields better results.

6. Conclusion

“Task 1 - Heuristic Search” was a challenging yet fun experience for our whole Team. Although there were some bumps along the way, like one Team-Member dropping out in the middle of implementation, initial and slight disagreements in the general structure of the software to be implemented and the difficulty of finding time to come together and discuss the project, while our jobs, families, studies and other deadlines required a part of our time and attention too, the experience in its totality was quite rewarding. It was satisfying to come to the end stage of the project, to see all the effort pay off and to finally, through the results of our own code implementation, find out which heuristic search function would come out on top.

Even though we gave our best in implementing the features as required in the Task description and feel confident in their fulfillment, there are still things which we could’ve done smarter. These could be part of a list of improvements for the future. Such things would be:

- **Object oriented approach** (abstraction for “Node”, “Tree” and similar, could possibly simplify the code)
- **Graphical User Interface**, instead of the overly simplified Command Line (User) Interface
- **More customization and user experimentation** (let the user create a custom start state, see a visual representation of the steps taken and nodes explored, create a sandbox for the user to explore the possibilities and workings of the algorithms in use)